Nombre: Carlos Drogba Zumba Hidalgo

Paralelo: A **Fecha: 13/10/2026**

Asignatura: Sistemas Operativos

Docente: Ing. Hernán Leonardo Torres Carrión M.Sc.

ENSAYO Nº 1

1. **Tema**

   Revisión literaria sobre los tipos de memoria caché y su funcionamiento, con énfasis en la caché a nivel de procesador y su importancia en la gestión de procesos.

2. **Antecedentes**

   La velocidad de los procesadores ha crecido mucho más rápido que la de la memoria principal (DRAM), lo que produce la llamada brecha procesador–memoria [1], [5]. Para reducirla se usa la memoria caché: una memoria pequeña y muy rápida, situada entre la CPU y la RAM, que guarda copias de los datos e instrucciones más usados [2], [6]. Funciona gracias al principio de localidad: temporal (lo usado recientemente tiende a usarse otra vez) y espacial (se suelen usar datos cercanos a los ya accedidos) [1], [2].

   Para la formación profesional en sistemas, este tema es clave porque el sistema operativo (SO) administra procesos e hilos sobre procesadores con varios núcleos y varios niveles de caché; decisiones como el cambio de contexto o la asignación de procesos a núcleos afectan directamente el rendimiento [3], [4]. Este conocimiento se aplica al optimizar programas, configurar servidores y máquinas virtuales y diagnosticar problemas de rendimiento. En este ensayo se resume la literatura principalmente mediante diagramas y un cuadro de elaboración propia.

3. **Descripción**

   3.1. Jerarquía de memoria y latencias

   Cuanto más cerca de la CPU, la memoria es más rápida, más pequeña y más costosa por bit (Fig. 1) [1], [2]. La Fig. 2 muestra el orden de magnitud de las latencias: un acierto en L1 tarda cerca de un nanosegundo, mientras que la RAM y el almacenamiento están varios órdenes de magnitud por encima [1], [5].

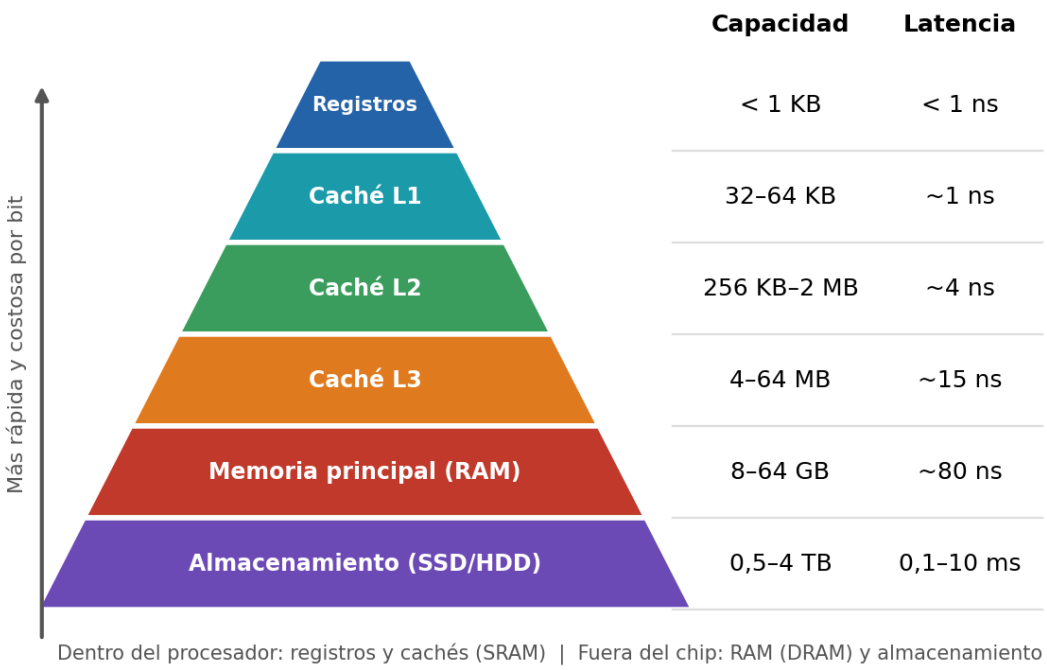


Fig. 1. Jerarquía de memoria de un computador (valores típicos aproximados). Elaboración propia a partir de [1], [2].

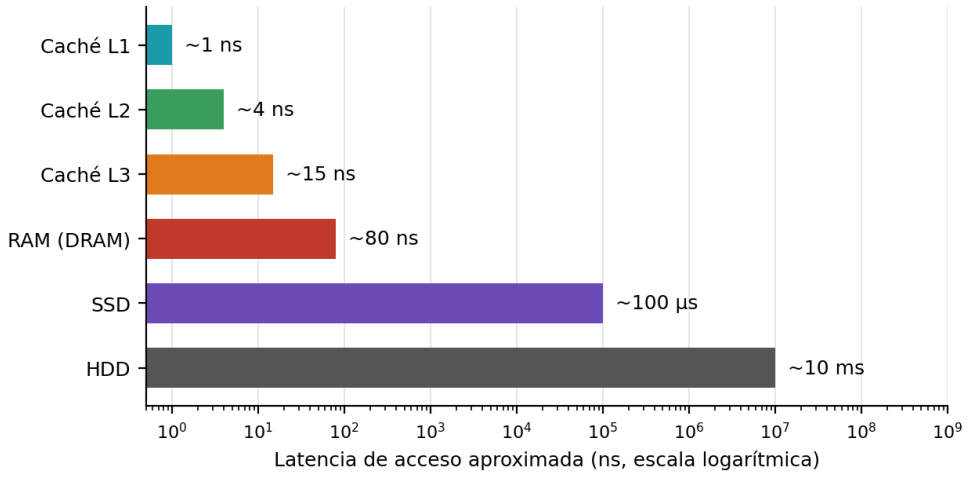


Fig. 2. Latencia aproximada de acceso por nivel de memoria (escala logarítmica). Órdenes de magnitud basados en [1], [5].

3.2. Tipos de memoria caché

Las cachés pueden clasificarse según varios criterios (Fig. 3) [2], [3]. Este ensayo se centra en las cachés de hardware del procesador; sin embargo, el SO también mantiene cachés en software (por ejemplo, la caché de páginas o de buffers en RAM), y el hardware incluye el TLB, una caché especializada en traducciones de direcciones virtuales a físicas [3], [4].

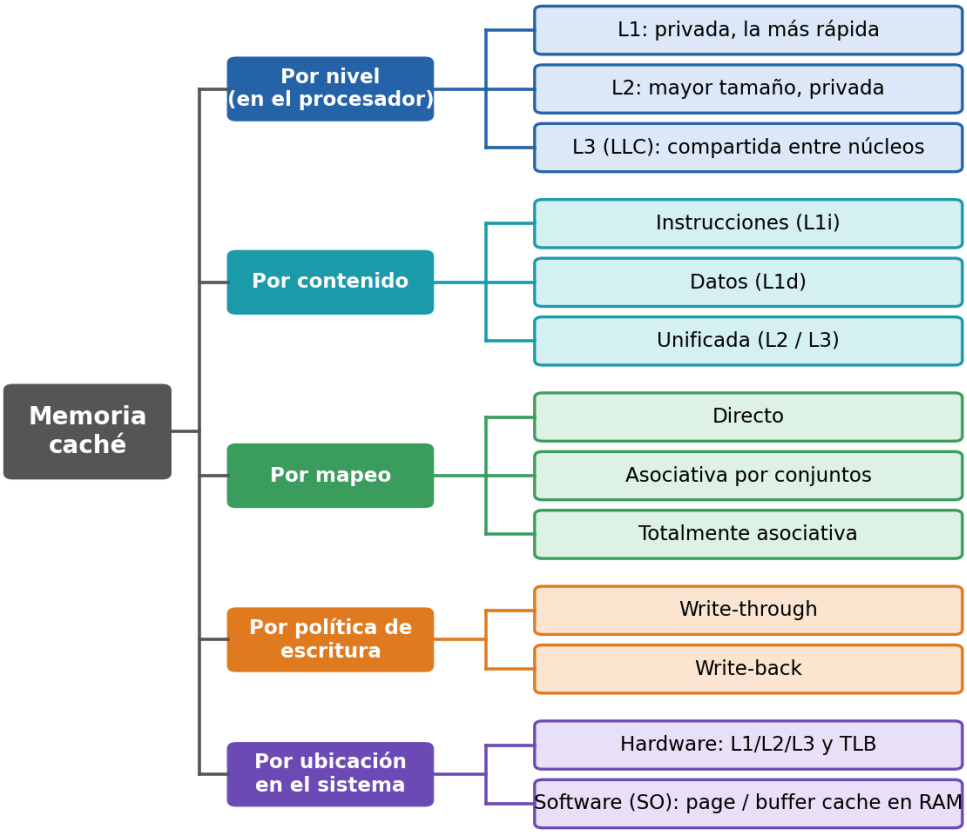


Fig. 3. Clasificación de la memoria caché según distintos criterios. Elaboración propia a partir de [2], [3].

3.3. Caché a nivel de procesador

En un procesador multinúcleo, la L1 es privada de cada núcleo y normalmente se divide en instrucciones y datos; la L2 es mayor y suele ser privada; la L3 es compartida y actúa como último nivel antes de la RAM (Fig. 4 y Tabla I) [1], [2].

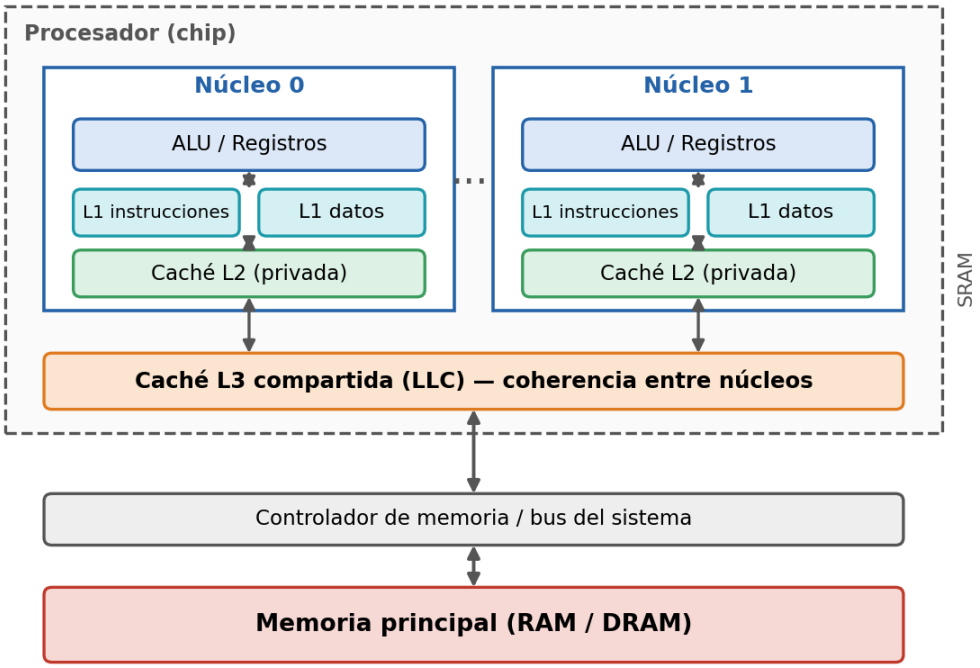


Fig. 4. Organización de la caché multinivel en un procesador multinúcleo. Elaboración propia a partir de [1], [2].

TABLA I

VALORES TÍPICOS APROXIMADOS DE LA JERARQUÍA DE CACHÉ

Nivel	Tamaño típico	Latencia típica	Alcance	Contenido
L1	32–64 KB por núcleo	~1–4 ciclos	Privada por núcleo	Instrucciones y datos (separadas)
L2	256 KB–2 MB	~10–15 ciclos	Normalmente privada	Unificada
L3 (LLC)	4–64 MB o más	~30–50 ciclos	Compartida entre núcleos	Unificada
RAM	Varios GB	~200 ciclos o más	Compartida por todo el sistema	Todo el sistema (DRAM)

Los valores varían según el modelo de procesador; fuente: adaptado de [1], [2].

3.4. Funcionamiento: aciertos, fallos, mapeo y reemplazo

Cuando la CPU pide un dato, el hardware lo busca en L1, luego en L2 y en L3. Si lo encuentra hay un acierto (hit); si no, un fallo (miss) y se accede a la RAM, trayendo una línea completa (por ejemplo, 64 bytes) para aprovechar la localidad espacial (Fig. 5) [1], [2].

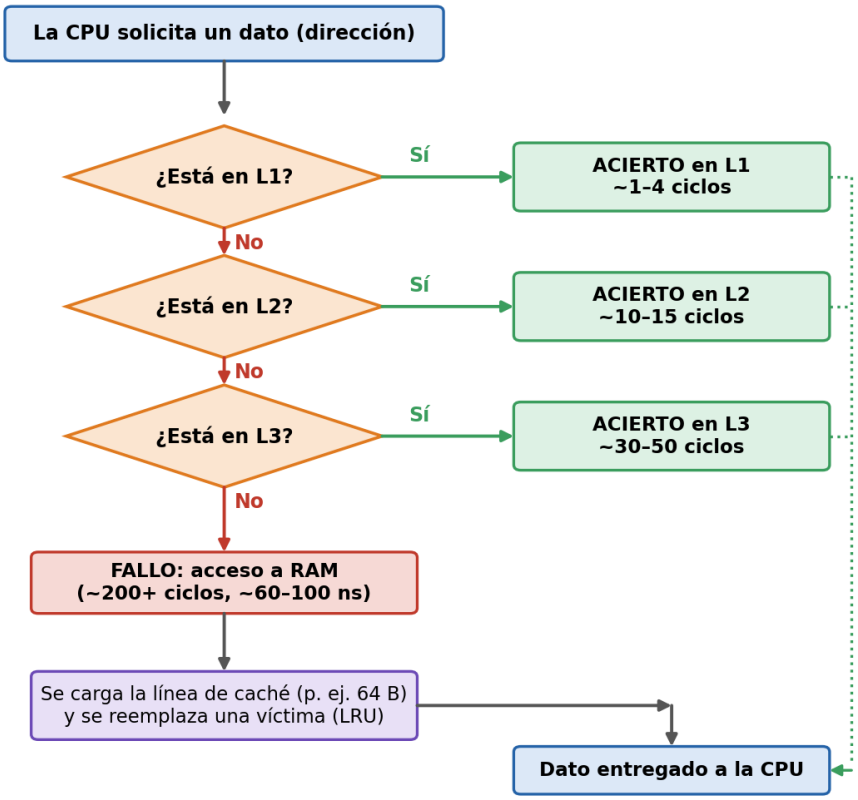


Fig. 5. Flujo de una lectura: acierto (hit) o fallo (miss) en la jerarquía de caché. Elaboración propia a partir de [1], [2].

El mapeo define en qué líneas puede guardarse un bloque (Fig. 6): directo (una sola línea posible), asociativo por conjuntos (varias líneas dentro de un conjunto) o totalmente asociativo (cualquier línea) [1], [2]. Si no hay espacio, una política de reemplazo (por ejemplo, LRU) elige la línea víctima, y la política de escritura (write-through o write-back) define cuándo se actualiza la RAM [2], [6].

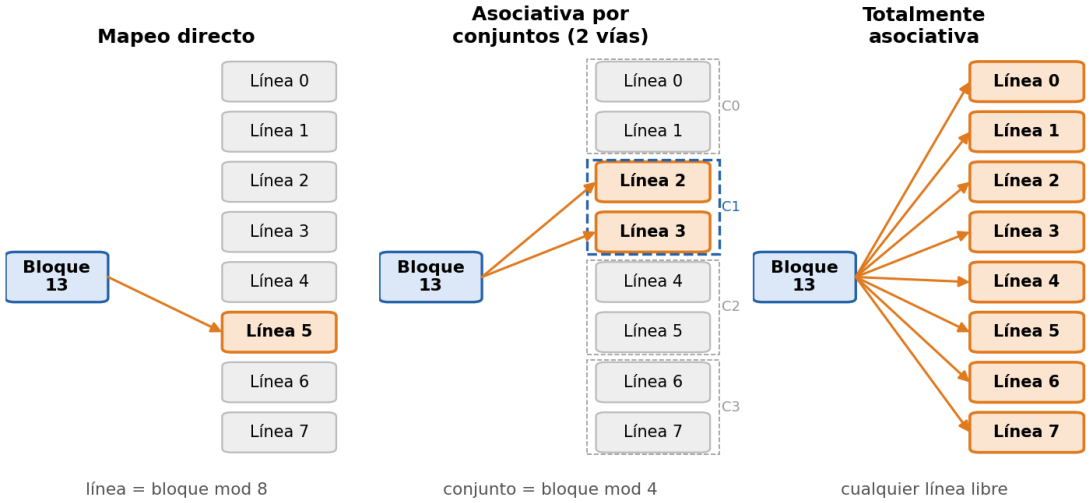


Fig. 6. Esquemas de mapeo de un bloque a líneas de caché (ejemplo: bloque 13, caché de 8 líneas). Elaboración propia a partir de [1], [2].

3.5. Coherencia entre cachés

En sistemas multinúcleo varias cachés pueden tener copias del mismo dato. Los protocolos de coherencia, como MESI, mantienen cada línea en uno de cuatro estados (Modificado, Exclusivo, Compartido o Inválido) para que todos los núcleos vean valores consistentes (Fig. 7) [2], [7].

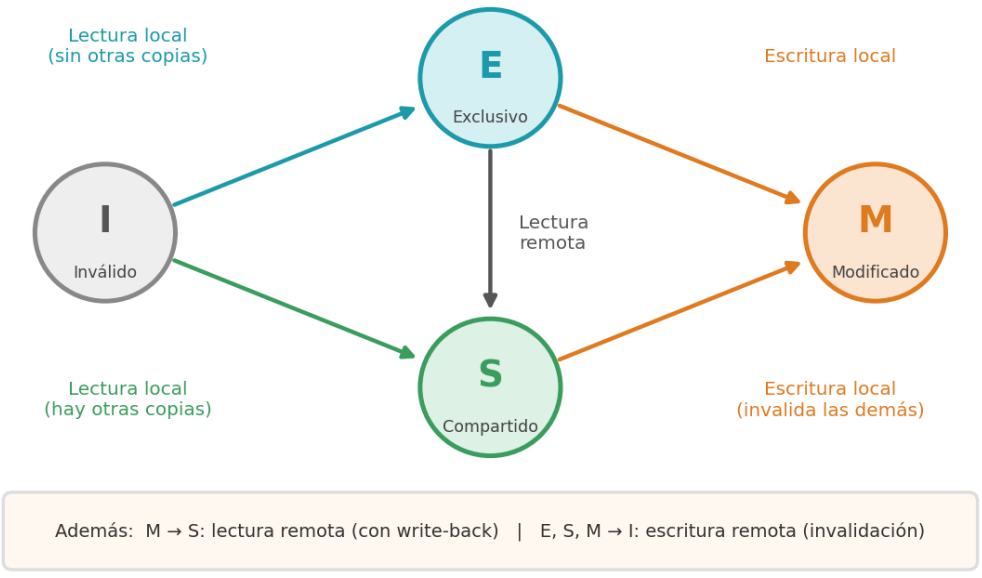


Fig. 7. Diagrama simplificado de estados del protocolo de coherencia MESI. Elaboración propia a partir de [7].

3.6. Importancia para la gestión de procesos

Aunque la caché la controla el hardware, el SO influye en su rendimiento mediante la planificación de procesos [3], [4]. Tras un cambio de contexto, el nuevo proceso encuentra una caché con datos del anterior y sufre una ráfaga de fallos hasta “calentarla” (Fig. 8); por eso un quantum demasiado corto puede perjudicar el rendimiento [3], [5].

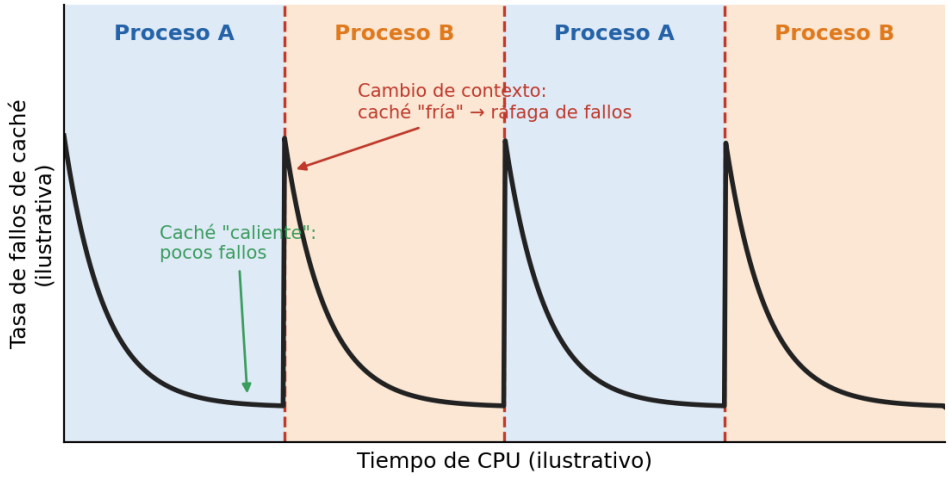


Fig. 8. Efecto conceptual del cambio de contexto en la tasa de fallos de caché (ilustración, no medición). Elaboración propia a partir de [3], [5].

Además, si el planificador migra un proceso a otro núcleo, este pierde los datos acumulados en la caché del núcleo anterior; por eso los SO modernos pueden favorecer la afinidad al procesador (Fig. 9) [3], [4]. También conviene considerar que el TLB puede requerir invalidación al cambiar de espacio de direcciones, salvo que use identificadores de espacio (ASID) [4], y que dos hilos que escriben en una misma línea de caché generan tráfico de coherencia innecesario (falso compartido) [5], [7].

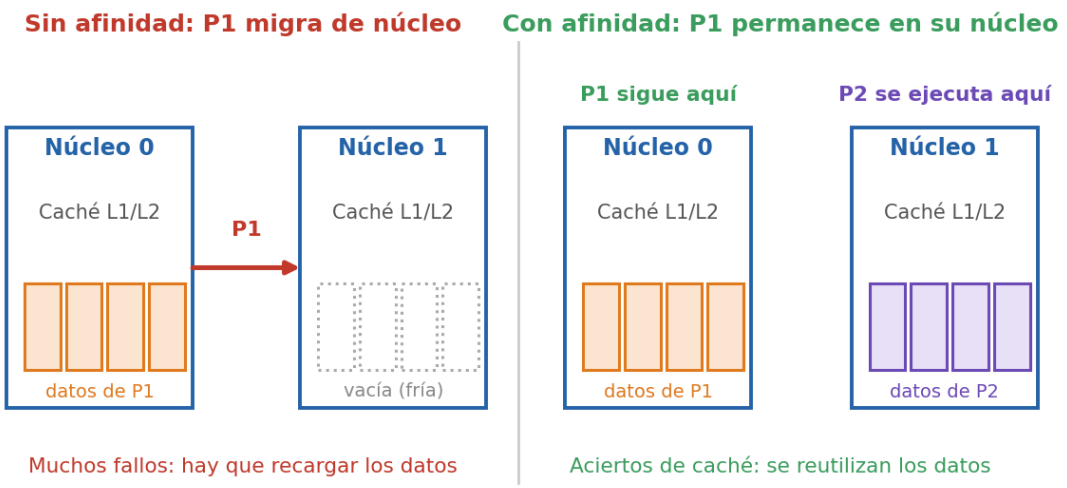


Fig. 9. Afinidad al procesador: migrar un proceso de núcleo descarta su caché caliente (ilustración conceptual). Elaboración propia a partir de [3], [4].

4. Conclusiones

Se ha concluido que:

- La memoria caché reduce la brecha entre la velocidad de la CPU y la de la RAM aprovechando la localidad temporal y espacial de los programas.
- En el procesador, la jerarquía L1–L2–L3 (privada y compartida) funciona mediante aciertos y fallos, mapeo, políticas de reemplazo y de escritura, y requiere protocolos de coherencia en sistemas multinúcleo.
- El SO no controla directamente la caché, pero sus decisiones de planificación (cambios de contexto, quantum y afinidad al procesador) determinan qué tan bien se aprovecha.
- Comprender estos conceptos ayuda a escribir programas más eficientes y a configurar y diagnosticar sistemas con mejor criterio.

5. Bibliografía

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.

[2] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 10th ed. Hoboken, NJ, USA: Pearson, 2016.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[4] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

[5] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., Raleigh, NC, USA, Nov. 2007.

[6] A. J. Smith, “Cache memories,” *ACM Comput. Surv.*, vol. 14, no. 3, pp. 473–530, Sep. 1982.

[7] D. J. Sorin, M. D. Hill, and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*. San Rafael, CA, USA: Morgan & Claypool, 2011.